

课题06-多卡与分布式

Contents

1 课题06 开题简报 · 多卡与分布式训练	1
1.1 一、研究背景	1
1.2 二、研究问题	1
1.3 三、攻关线索	2
1.4 四、里程碑	2
1.5 五、交付物	2
1.6 六、讲课锚点	2
1.7 七、代码级提问示例	3
1.8 八、风险与提醒	3

1 课题06 开题简报 · 多卡与分布式训练

模块: M4 | 周次: W9 (11-09→11-15) | 算力: T2 Kaggle 2×T4, 计划 8 GPU·小时 唐杰作业对应: ② 的”多卡训练增益”部分 | 判分来源: CS336 A2 分布式部分 + 自建 benchmark 手写: R3 的延伸 (并行策略选择与通信原语判断)

1.1 一、研究背景

数据并行 (DDP) 把同一模型复制到每张卡、各自算一个微批、再同步梯度; 当模型放不下单卡时, 才有 FSDP/ZeRO (切分参数、梯度、优化器状态)、张量并行 (切层内矩阵)、流水线并行 (切层)。Kaggle 免费档给的是 2×T4, 且走 PCIe 而非 NVLink —— 这恰好是”观察通信成为瓶颈”的最佳实验台。

1.2 二、研究问题

在弱互联 (PCIe) 的 2×T4 上, DDP 的扩展效率是多少? 瓶颈是计算还是通信? FSDP 在什么规模才开始划算?

1.3 三、攻关线索

1. 先对齐正确性：DDP 后的梯度应与单卡（同等效批大小）数值一致，先验证再谈性能
2. 批大小语义：2 卡 $\times$ 每卡 batch = 等效批大小；若显存受限要同步调整学习率（线性/平方根缩放）
3. 计时口径：step time 要区分“纯计算时间”与“含 all-reduce 的时间”；用 torch.profiler 或手写计时
4. 扩展效率： $E = \frac{T_1}{2T_2}$ 。先预测（PCIe 下 0.6-0.8?）再测
5. 通信占比：梯度同步量与参数量成正比（ $2N$ 字节/步，all-reduce 近似）；序列长/批小时通信占比更高
6. **FSDP** 的代价：切分带来额外通信（all-gather + reduce-scatter），小模型上往往更慢

1.4 四、里程碑

- ☐ M1 DDP 梯度一致性验证（与单卡同等效批） $\rightarrow$ 记录最大误差
- ☐ M2 单卡 vs 双卡的 step time、tokens/s 对照（同模型、同等效批）
- ☐ M3 扩展效率曲线（至少 3 个配置点：不同 batch/序列长度）
- ☐ M4 通信占比分解：计算 / all-reduce / 数据加载 三段计时
- ☐ M5 一次 FSDP 实验并解释“为什么（不）划算”
- ☐ M6 先写预测再实测：预测扩展效率，跑完对照（进 memory/MEMORY.md 打脸日志）
- ☐ M7 一页报告

1.5 五、交付物

交付物	位置	验收
分布式脚本	手写/ 或 脚手架/（按 B3 分类）	可复跑
计时原始数据	证据/bench.csv	含每配置 3 次重复
扩展效率曲线	证据/	标注硬件（2 $\times$ T4 PCIe）与对比基准
报告	报告.md	解释“为什么不是 2 倍”

1.6 六、讲课锚点

- 讲义：M4-训练.md（W3 展开）中并行一节
- 对标：CS336 lecture 07-08（Parallelism） $\cdot$ nanochat/optim.py（分布式优化器实现） $\cdot$ HF Ultra-Scale Playbook（用前先核）
- 搭桥：M0 的“梯度累加” $\rightarrow$ DDP 就是跨卡的梯度累加

1.7 七、代码级提问示例

1. 2×T4 上 DDP 只快了 1.4 倍，列出三个最可能的原因。
2. 把每卡 batch 减半、用梯度累积补回等效批，通信量会怎么变？
3. 什么情况下 FSDP 会比 **DDP** 慢？（用通信量解释）
4. all-reduce 的通信量是多少字节/步？用参数量 N 表示。

1.8 八、风险与提醒

- △ Kaggle 双 T4 会话不稳定（可能只给一张）：预检时确认 `torch.cuda.device_count()==2`。
- △ 不要在本课题纠缠“调到最快”——目标是理解扩展效率的成因，配额要留给课题08。